

Day 2 Lab — Token Validation, Sender-Constraining & Request Hardening

JWT/JWKS validation · DPoP · PAR + JAR

Goal of This Lab

Day 1 taught the OAuth **flows**. Day 2 teaches what makes them **trustworthy**. By the end you should be able to:

- Validate a JWT access token yourself — signature, issuer, expiry, scope — against a JWKS, and explain why decoding a token is not the same as verifying it.
- Explain how a **DPoP** sender-constrained token defeats token replay, and what a stolen token is worth without the client's key.
- Explain why **PAR** and **JAR** move the authorization request off the browser and make it tamper-proof.

You run each example; you do not need to write code. Keep the terminal beside the browser — the per-request logs show which actor talks to which endpoint.

Setup

Requires Go 1.21+ (pinned to 1.25). Dependencies are vendored; no Docker, no network. Every example starts three servers — Authorization Server (:8080), Resource Server (:8081), Client (:8082) — and you enter at **http://localhost:8082**.

```
cd resource/labs/day2-oauth-demo
go run ./cmd/jwt-validation      # Exercise 1
go run ./cmd/dpop                # Exercise 2
go run ./cmd/par-jar             # Exercise 3
go run ./cmd/fapi                # Exercise 4
```

Run one at a time (they share the same ports). Stop one with Ctrl-C before starting the next.

Exercise 1 — JWT Validation (Lessons 4 & 5)

```
go run ./cmd/jwt-validation
```

The idea: a JWT access token is self-contained. The Resource Server verifies it locally against the Authorization Server's published JWKS — no introspection round-trip.

Steps

1. **Run Authorization Code + PKCE.** The token page decodes the JWT: header, payload, signature. Note the payload is readable Base64URL — anyone can read it.
2. **Call resource API (valid token)** → 200. The Resource Server fetched the AS's public keys from `/jwks` and verified the signature, `iss`, `exp`, and scope itself.
3. **Tamper with the token** → the client flips one payload byte without re-signing. The API returns 401 `signature check failed` — caught locally, with no call to the AS.

Questions

- If anyone can read a JWT payload, why is it safe to send? What makes it *trusted*?
 - The Resource Server never contacts the AS to validate. What does it gain, and what does it give up (revocation, key rotation)?
 - Why must the server pin the accepted algorithm (RS256) rather than trust the token's own `alg` header?
 - When would you still prefer introspection (the Day 1 approach)?
-

Exercise 2 — DPoP Sender-Constrained Tokens (Lesson 7)

```
go run ./cmd/dpop
```

The idea: the access token is bound to a key the client holds (`cnf.jkt`). Every API call must carry a fresh **DPoP proof** signed by that key, so a stolen token alone is useless.

Steps

1. **Run Authorization Code + PKCE (with DPoP).** The token request carries a DPoP proof; the AS binds the issued token to the proof's key. Note `"token_type": "DPoP"`.

2. **Call API with token + DPoP proof** $\rightarrow$ 200. The Resource Server checks the JWT *and* that the caller proved possession of the bound key for this exact method + URL + token.
3. **Replay the token WITHOUT the key** (presented as a plain Bearer, no proof) $\rightarrow$ 401. This is the attacker who exfiltrated the token but not the private key.

Questions

- What is in the DPoP proof (**htm**, **htu**, **ath**, the embedded **jwk**), and why is each needed?
- How does **cnf.jkt** in the access token tie back to the proof's key?
- DPoP defends a *leaked token*. What does it **not** defend against, and would mTLS differ?
- Why must each proof be fresh (bound to method, URL, and token hash)?

Exercise 3 — PAR + JAR: Hardening the Request (Lesson 7)

```
go run ./cmd/par-jar
```

The idea: the client sends its authorization request as a **signed request object (JAR)**, **pushed** to the AS back-channel first (**PAR**). The browser only carries an opaque **request_uri**; the parameters are integrity-protected and never exposed or tamperable.

Steps

1. **Build request object** $\rightarrow$ **PAR** $\rightarrow$ **authorize**. The page shows the decoded, signed request object and the PAR response. Notice the **/authorize** URL carries only **client_id**, **response_type**, **scope**, and the opaque **request_uri** — no redirect URI or PKCE challenge is exposed in the browser.
2. **Continue to /authorize**, approve, and the token is issued as usual.
3. **Negative check (instructor):** a plain **/authorize** request without PAR is rejected (400), because this AS requires PAR + a signed request object.

Questions

- What can leak or be tampered with in a classic front-channel authorization URL? How do PAR and JAR each remove that risk?
- The AS verifies the request object against the client's *registered public key*. Why does that matter, and what does the client prove by signing it?

- `response_type/scope` still appear as plain params for OpenID requests. Why is that safe when the signed, pushed request is the source of truth?
-

Exercise 4 — FAPI 2.0 Capstone (Lesson 8)

```
go run ./cmd/fapi
```

The idea: the FAPI 2.0 baseline profile combines the controls above into one hardened configuration. `WithProfile(FAPI2)` makes the Authorization Server *reject* any setup that is not compliant, so this example is essentially the minimum FAPI 2.0 demands: `private_key_jwt` client authentication + PAR + PKCE (S256) + DPoP-sender-constrained tokens + short-lived authorization codes.

Steps

1. **Assertion** → **PAR** → **authorize**. The page shows the decoded `private_key_jwt` client assertion (the client authenticates with a signed JWT, no secret). It is pushed to `/par`, and the browser continues with only an opaque `request_uri`.
2. **Approve**, and the token is issued — note `"token_type": "DPoP"`. The client authenticated with the assertion *and* proved possession of its DPoP key.
3. **Call API** → 200, “FAPI 2.0: JWT verified locally AND DPoP proof-of-possession confirmed”. Every layer — client auth, request integrity, sender-constraining — is active.

Questions

- Which specific attack does each FAPI 2.0 control stop? Map `private_key_jwt`, PAR, PKCE, and DPoP to a threat.
 - Why does FAPI forbid public clients and client secrets in favour of `private_key_jwt`?
 - FAPI 2.0 requires *sender-constrained* tokens (DPoP **or** mTLS). Why is a plain Bearer token never acceptable for a high-value API?
-

Paper-Only Topics (no runnable demo here)

The lesson papers cover three more controls that these labs do not demonstrate as code, to keep each example focused. Point participants to the papers for:

- **JARM** (JWT-Secured Authorization Response) — Lesson 7 §5.
- **mTLS** sender-constraining — Lesson 7 §9 (an alternative to DPoP).
- **JWE** (encrypted JOSE) — Lesson 5.

Wrap-up Discussion

- Map each control to the threat it addresses: JWT/JWKS validation, DPoP, PAR, JAR.
- Exercise 4 combined them into the FAPI 2.0 baseline. Which of these controls would you adopt first on a system you work on, and why?
- Where would mTLS be a better sender-constraining choice than DPoP, and vice versa?